Week 4

1. Arrays

→ Homogeneous collection of elements that are of the same type
→ Finite
→ Continuous memory locations
→ Static, size is pre-defined and immutable

2. Lists (uses '[]' square brackets)

→ e.g. Labels = ['R-001', 'R-002', 'R-003']

print(Labels[0])

↳ gets element one ('R-001')

→ Heterogeneous collection of elements

→ supports reverse indexing:

```
[5, 10, 15]
 ↓   ↓   ↓
-3  -2  -1
```

→ supports slicing, i.e. extracting certain values only:

```
L = [5, 10, 15, 20, 25]

L[1:-1]

↳ prints [10, 15, 20]
```

→ Lists are mutable, the values can be changed

→ traversed using for loop:

```
L = [5, 10, 15]

for element in L:
    print(element)
```

3. Tuples (uses '()' brackets)

→ immutable. (cannot be changed.

```
point = (3, 7)
point[0] = 4
```

↳ TypeError: 'tuple' object does not support object assignment

4. List comprehension:

values = [5, 10, 15, 20, 25]

doubled = [v * 2 for v in values]

5. Dictionaries:

→ stores a key: value pair

e.g.: {'name': 'Sandesh'}

'name' → key

'Sandesh' → value

dictionary with one element (one key: value pair)

→ values accessed with corresponding key:

```
L = {'name': 'Sandesh'}
print(L['name'])
```

↳ prints Sandesh

```
print(L[0])
```

↓

standard integer indexing does NOT work.

→ adding a key:value pair:

```
L = {'name': 'Sandesh'}
L['age'] = 18
```

↳ new element added

→ modifying a value

```
L['name'] = "San P"
```

↳ modified to 'San P'

→ dict.items() returns a list of all (key, value) pairs

6. zip()

function to combine two lists by pairing up the individual elements with each other. Example:

```
hostnames = ["srv1", "srv2", "srv3"]
disk_used = [87, 45, 120]

paired = list(zip(hostnames, disk_used))
print(paired)
```

↳ prints [("srv1", 87), ("srv2", 45), ("srv3", 120)]

7. Sets

→ mutable
→ ONLY unique values
→ made using set() function or '{}' (curly braces).
→ NOT {} → thats an empty dictionary, not set

8. Error Handling in Python

If errors are not handled; they can stop program execution flow.

To handle an error, we use the try / except block in python.

For example:

```
try: — (1)
    with open("missing.txt") as f:
        text = f.read()
except FileNotFoundError: — (2)
    print("file not found")
    text = ""

print(text)
```

(1) → the error prone block of code is placed in the try block

(2) → the except block is where the error is handled. In the case, the file not found error is handled.